

24-Hour AI Development Challenge

AI-Powered Adaptive Self-Learning Platform

Build a working AI-first learning product that assesses a learner, adapts to performance, and demonstrates meaningful AI-driven decision making.

Duration	24 hours
Domain	Artificial Intelligence
Assignment Type	Individual development challenge
Final Output	Working, deployed MVP + source code + documentation

Expected outcome

A working MVP that completes the core learner journey from assessment to personalized learning, evaluation, weakness detection, and adaptive remediation. Deployment, persistence, dashboard depth, and other extended capabilities are Optional / Pending for the 24-hour submission.

1. Assignment Objective

Build and deploy an AI-powered self-learning platform that can understand a learner's current level, generate a personalized learning path, teach AI concepts, evaluate understanding, identify weak areas, and adapt what the learner should study next.

The solution must behave differently for different learners. A fixed course sequence, static LMS, or generic chatbot is not sufficient.

Learner Profile -> Diagnostic Assessment -> Skill Analysis -> Personalized Path -> Learn -> Assess -> Detect Weakness -> Adapt -> Continue

2. Domain Scope

The platform must focus only on the AI domain. You may support one or more learning tracks, but the selected track must demonstrate the complete adaptive learning flow.

- Python for AI
- Machine Learning
- Deep Learning
- Natural Language Processing
- Generative AI
- Large Language Models
- Retrieval-Augmented Generation (RAG)
- AI Agents / Agentic Systems

You are not expected to build complete course content for every track. Depth and working behaviour matter more than the number of topics.

3. Expected Learner Journey

Stage	Expected Result
1. Learner Profile	Capture the learner name/profile, current experience level, and learning goal.
2. AI Goal Selection	Allow the learner to select or define a learning goal within the AI domain.
3. Diagnostic Assessment	Generate or present an initial assessment that measures current knowledge.
4. Skill Analysis	Convert assessment performance into useful skill-level information, not just one overall score.
5. Personalized Learning Path	Generate a learning sequence based on the learner's goal, strengths, weaknesses, and prerequisites.
6. Learning Experience	Provide concept explanations, examples, practical exercises, and access to an AI tutor.
7. Assessment	Evaluate the learner after topics or modules using suitable question types.
8. Weakness Detection	Identify the exact concept or area where the learner is struggling.
9. Adaptive Response	Change explanation, practice, difficulty, or next-topic recommendation according to the learner's performance.
10. Progress Persistence	Store progress so that refresh/reopen does not reset the learner state.

4. Personalized Learning Path Behaviour

The learning path must not be a manually fixed list that is identical for every learner. The platform should use the learner's goal, current knowledge, prerequisite gaps, and assessment history when deciding the next learning step.

Expected behaviour example

Goal: Generative AI Developer. Diagnostic result: Python 80%, Machine Learning 60%, Deep Learning 30%, Transformers 10%, LLMs 5%. A suitable path may prioritize Deep Learning Fundamentals -> Neural Networks -> Attention -> Transformers -> LLM Fundamentals -> Prompt Engineering -> RAG -> AI Agents. A stronger learner should receive a different path.

The exact path-generation strategy is your technical decision. You should be able to explain why the system generated a particular path.

5. Adaptive Learning and Weakness Recovery

The platform must react to learner performance. The learner should not always move forward after completing a quiz.

Expected behaviour example

A learner scores Attention 90%, Self-Attention 85%, Positional Encoding 35%, Encoder/Decoder 70%. The platform should identify Positional Encoding as a weak concept, provide targeted remediation, and reassess or modify the next recommendation before continuing normally.

- Identify weak concepts at a useful level of detail.
- Provide an alternate explanation, targeted practice, or revision when needed.
- Allow strong performance to increase difficulty or move the learner forward.
- Update learner state after each meaningful activity.
- Make the next recommendation depend on actual learner performance.

6. Assessment Requirements

For the 24-hour MVP, one working assessment format is sufficient. Additional formats listed below are part of the extended scope (Optional / Pending).

Question Type	Expected Use
MCQ	Concept or application-based questions with plausible distractors.
Conceptual	Ask the learner to explain a concept in their own words.
Scenario-based	Require a model, approach, or design choice based on a practical situation.
Code Output	Ask the learner to predict or interpret Python/AI code behaviour.

Question Type	Expected Use
Coding	Ask for a small implementation relevant to the topic.
Debugging	Present incorrect code or logic and ask the learner to identify or fix the issue.
Comparison	Compare two AI concepts, methods, models, or architectures.

Difficulty variation is Optional / Pending for the MVP. The mandatory adaptive behaviour can instead be demonstrated through targeted remediation or the next-topic recommendation.

7. Context-Aware AI Tutor

Extended feature (Optional / Pending for the 24-hour MVP): The AI tutor should use learner context. At minimum, it should know the learner's current topic, completed topics, and relevant weak areas. A generic chatbot embedded in the interface will not satisfy this requirement.

Expected behaviour example

If the learner has completed Python, NumPy, Pandas, and Machine Learning Basics, an explanation of Transformers should account for that background instead of responding as if the learner has no prior knowledge.

8. "Teach Me Differently" Feature

Extended feature (Optional / Pending for the 24-hour MVP): Provide a visible way for a learner to indicate that an explanation was not understood. The next explanation must change teaching style rather than repeat the same response.

- Simplified explanation
- Real-world analogy
- Technical explanation
- Code-based explanation
- Step-by-step explanation

You may use different labels or interaction design, but the change in teaching approach must be clear.

9. AI Engineering Requirements

Use an LLM or comparable generative AI capability for meaningful parts of the product. Suitable uses include learning-path generation, explanation, quiz generation, answer evaluation, weakness analysis, recommendations, or tutor interaction.

- Do not rely on the LLM for every decision. Deterministic application logic should be used where appropriate, especially for scoring, state management, validation, and progress tracking.

- Extended feature (Optional / Pending for the 24-hour MVP): At least one AI-driven feature should produce validated structured output such as a schema-validated JSON object or data model.
- For the extended structured-output feature, the application should handle malformed responses instead of assuming every AI response is valid.
- Your prompts, schemas, validation strategy, and AI orchestration should be understandable from the codebase.

Structured output example

topic: Transformers | difficulty: intermediate | prerequisites: Neural Networks, Attention | weak_areas: Self-Attention | recommended_next_topic: Multi-Head Attention

10. Reliability and Content Quality

Educational content should not be trusted only because an LLM generated it. Implement at least one practical method to improve reliability or grounding.

Possible approaches include curated content, retrieval, trusted documentation, source grounding, citations, structured validation, or another defensible method. The exact implementation is intentionally left open.

11. Failure Handling

Failure handling is part of the extended scope and is Optional / Pending for the 24-hour MVP. If implemented, the application should avoid raw exceptions, silent failures, or completely broken screens.

- AI API failure or timeout
- Empty model response
- Malformed or invalid structured output
- Unexpected values returned by the model
- Temporary backend or network failure

A raw exception, silent failure, or completely broken page should be avoided.

12. State, Memory, and Data Persistence

Extended feature (Optional / Pending for the 24-hour MVP): Persist learner state so progress can be resumed and adaptive decisions can use previous activity. If implemented, the data model may account for the following areas where relevant:

Data Area	Purpose
Users / Profiles	Learner identity and baseline information.
Learning Goals	Selected target, pathway, or specialization.

Data Area	Purpose
Learning Path	Generated modules, ordering, prerequisites, and current position.
Topic Progress	Completed, in-progress, and pending topics.
Assessment Results	Scores, question-level performance, and attempts.
Weak Concepts	Detected gaps and remediation status.
Question History	Previously asked questions or identifiers to reduce repetition.
Tutor / Learning Context	Relevant learner context required for AI interactions.
Recommendations	Current next action and why it was selected.

13. Required Challenge Conditions

Condition	Expected Result
No Fixed Path	Mandatory: Different learners or different assessment results should be able to produce different paths or recommendations.
No Duplicate-Question Loop	Optional / Pending: Track question history or use another strategy to reduce repeated questions.
Goal Change	Optional / Pending: If the learner changes the target goal, reuse relevant completed learning instead of blindly resetting everything.
Structured AI Output	Optional / Pending: At least one important AI response must be validated before the application uses it.
Persistence	Optional / Pending: Persist meaningful learner progress after refresh/reopen if time permits.
AI Failure	Optional / Pending: The user should receive a controlled fallback or recovery path when an AI request fails.
Real Data Flow	Optional / Pending: If a dashboard is implemented, values should come from actual learner activity rather than static data.

14. Technical Direction

You may choose your own stack. Technology choice by itself is not the objective. Select tools that allow you to produce a stable, testable product within 24 hours.

Layer	Possible Options
Frontend	React, Next.js, or equivalent
Backend	FastAPI, Node.js, or equivalent
Database	PostgreSQL, MongoDB, SQLite for MVP, or equivalent
AI Provider	OpenAI, Gemini, Claude, Ollama, or another suitable model
Retrieval / Vector Layer	FAISS, Chroma, Pinecone, pgvector, or equivalent if required
Deployment	Vercel, Render, Railway, cloud VM, or equivalent

These are examples only. You are free to choose another architecture if you can justify it.

15. Progress Dashboard Expectations

Extended feature (Optional / Pending for the 24-hour MVP): A progress dashboard may show learner progress and assessment information using real application data rather than static values.

- Overall progress
- Completed topics or modules
- Current topic
- Strong areas
- Weak areas
- Recent or average assessment performance
- Recommended next topic or action
- Recent learning activity or history

16. Minimum Working MVP Expected at the End of 24 Hours

MVP priority: Exactly 8 core items are Mandatory for the 24-hour submission. Everything else in this document is Optional / Pending and may be implemented after the first 24 hours. Optional items must not block MVP acceptance. For learning and assessment, one AI topic track and one working question type are sufficient.

Requirement	24-Hour MVP Priority
Learner profile and AI-domain goal selection	Mandatory
Diagnostic assessment	Mandatory
Skill analysis	Mandatory
Personalized learning path	Mandatory

Requirement	24-Hour MVP Priority
AI-generated or AI-assisted learning content	Mandatory
Assessment after learning	Mandatory
Weak-topic identification	Mandatory
Adaptive recommendation or remediation	Mandatory
Context-aware AI tutor	Optional / Pending
Progress dashboard	Optional / Pending
Persistent learner state	Optional / Pending
Validated structured AI output	Optional / Pending
Basic error handling	Optional / Pending
Deployed application	Optional / Pending

17. What Will Not Be Considered a Complete Solution

- A static course website with manually fixed modules and no adaptation.
- A normal chatbot with a course-themed prompt.
- An application that only sends prompts to an LLM and displays the returned text.
- A UI-only prototype where major buttons or flows do not work.
- Hard-coded analysis, progress, or recommendations presented as dynamic results.
- A dashboard containing dummy data unrelated to actual learner actions.
- Lack of persistence after refresh or restart is acceptable for the 24-hour MVP, but persistence is preferred as an Optional / Pending enhancement.
- A local working demo is acceptable for the 24-hour MVP. Deployment is Optional / Pending, but is preferred if time permits.

18. Final Submission Requirements

Deliverable	What to Submit
Working Application	A working local or deployed application that can be demonstrated. Deployment is Optional / Pending.
Source Code	Complete repository with readable project structure.
README	Setup steps, architecture overview, feature list, environment variables, and run

Deliverable	What to Submit
	instructions.
Architecture Summary	Short explanation of frontend, backend, database, AI layer, and data flow.
AI Design Notes	Where AI is used, where deterministic logic is used, and how structured outputs are validated.
Known Limitations	Anything incomplete, simplified, or intentionally scoped for the 24-hour build.
Demo Evidence	Be ready to demonstrate the complete learner journey live.

19. Final Demo Scenario

During review, your application should be able to demonstrate the following journey without manually editing code or database records:

1. Create or select a learner profile and define an AI learning goal.
2. Complete the initial diagnostic assessment.
3. Show the resulting skill analysis and personalized learning path.
4. Open a learning topic and use the learning content; demonstrate an AI tutor if implemented.
5. Complete an assessment that includes at least one weak result.
6. Show how the platform identifies the weak concept.
7. Show the remediation, changed explanation, changed difficulty, or changed recommendation.
8. Continue or reassess and show the learner state updating.
9. If persistence is implemented, refresh or reopen the application and show that learner progress is retained.

20. Submission Notes

Prioritize a working end-to-end product over a large number of partially implemented features. The final submission should make the mandatory adaptive behaviour visible through real learner actions and stored data.

Do not commit API keys, passwords, or other secrets to the source repository. Include an example environment configuration where required.

END OF ASSIGNMENT